

CSCE 611 OPERATING SYSTEMS · MODULE 05 · LECTURE NOTES

Deadlock

Four conditions, and four ways to attack them.

MODULE OUTCOMES

State the four necessary conditions for deadlock.

Detect deadlock with a wait-for graph.

Apply prevention, avoidance, detection, and the ostrich algorithm.

Explain lock ordering and why it is the practical answer.

Distinguish deadlock from livelock and starvation.

1 The four conditions

Coffman's conditions are **necessary**: deadlock requires all four simultaneously. That makes them a checklist for defence — eliminate any one structurally and deadlock becomes impossible rather than unlikely.

Condition	Statement	How it could be broken
Mutual exclusion	At least one resource is held in a non-shareable mode.	Use lock-free data structures, or immutable data that needs no exclusion. Rarely available in general.
Hold and wait	A thread holding a resource requests another.	Acquire everything needed in one atomic step, or release everything before requesting more. Hurts concurrency and is often impractical.
No preemption	A resource cannot be forcibly taken from its holder.	Use trylock and release on failure; or allow rollback. Databases do this.
Circular wait	A cycle exists in the wait-for graph.	Impose a global ordering on locks and always acquire in increasing order. This is the one used in practice.

```
// Thread A
acquire(&lock1);
acquire(&lock2);
...
release(&lock2);
release(&lock1);

// Thread B
acquire(&lock2);
acquire(&lock1);
...
release(&lock1);
release(&lock2);
```

Each thread is individually correct: it acquires what it needs, does its work, and releases in reverse order. The defect exists only in the *interaction*, which is why reviewing either function alone cannot find it and why deadlocks survive careful code review.

2 Lock ordering: the practical answer

BREAK CIRCULAR WAIT AND THE REST DOES NOT MATTER

Define a total order over all locks. Require every thread to acquire locks in strictly increasing order.

For a cycle to form, some thread must be holding a higher-ordered lock while requesting a lower-ordered one. If the discipline is never violated, that cannot happen, so no cycle can form, so deadlock is **structurally impossible** — not merely unlikely.

The run-time cost is zero. The cost is discipline, and discipline is enforceable: a debug build can record the highest-ordered lock each thread holds and assert when a lower one is requested. Linux's **lockdep** does exactly this, and it reports violations even on runs where no deadlock actually occurred.

```
// Ordering by address: for many interchangeable objects
void transfer(Account *a, Account *b, int amount) {
    Account *first = (a < b) ? a : b;
    Account *second = (a < b) ? b : a;
    acquire(&first->lock);
    acquire(&second->lock);
    a->balance -= amount;
    b->balance += amount;
    release(&second->lock);
    release(&first->lock);
}

// Ordering by level: for layered subsystems
// 1. filesystem lock
// 2. inode lock
// 3. buffer cache lock
// Never acquire level N while holding level > N.
```

Address ordering handles the common case of many structurally identical objects — accounts, inodes, connections — where no natural hierarchy exists. Level ordering handles layered subsystems, and has the advantage of being documentable in a way humans can follow.

Write the order down, in the header that declares the locks. An undocumented ordering is one that a new contributor will violate within a month, and the resulting bug will appear six months later under load.

3 The four strategies

Strategy	Approach	Assessment
Prevention	Make one of the four conditions structurally impossible.	In practice this means lock ordering. Free at run time, enforceable in debug builds. The right default.
Avoidance	Analyse each request and refuse any that could lead to an unsafe state (Banker's algorithm).	Requires each process to declare its maximum future resource needs in advance. No real program can. Not used in any general-purpose OS.
Detection and recovery	Allow deadlock; periodically search the wait-for graph for cycles; break them.	Requires a way to recover, which means killing or rolling back a participant. Databases can; operating systems generally cannot.
The ostrich algorithm	Ignore the possibility. If the system hangs, reboot.	What general-purpose operating systems actually do , and a defensible engineering decision: deadlocks are rare, detection is expensive, and recovery is often impossible anyway.

ON THE BANKER'S ALGORITHM

Textbooks devote considerable space to deadlock avoidance, and it is worth understanding the idea of a **safe state** — one from which some completion order exists for all processes.

But the algorithm requires advance declaration of maximum resource needs, which no real program can supply, and it costs an $O(n^2m)$ safety check on every request.

It is not used in any general-purpose operating system. Knowing why it is impractical is more useful than being able to execute it.

3.1 Where detection is right

- **Databases** detect and recover, because the machinery already exists: a transaction can be aborted and retried, and users expect that. The system builds a wait-for graph among transactions, finds cycles, and aborts the cheapest victim — typically the one holding the fewest locks or having done the least work.
- **Operating systems** generally cannot, because killing a process in the middle of a kernel operation leaves data structures half-updated with no way to roll back.
- **Debugging tools** are the important exception: lockdep in the Linux kernel and ThreadSanitizer in user space detect *ordering violations*, which are evidence of a potential deadlock, even on executions where no deadlock occurred. This is far more useful than detecting the deadlock itself, because it finds the bug before a customer does.

4 Deadlock, livelock, starvation

Failure	Behaviour	CPU use	Cause
Deadlock	Threads blocked permanently in a cycle.	Zero — conspicuous.	All four Coffman conditions hold.
Livelock	Threads run continuously and accomplish nothing.	100% — looks healthy.	Symmetric detect-and-retry with no backoff.
Starvation	One thread never obtains a resource while others proceed.	Normal.	An unfair policy: continuous readers blocking a writer, or a scheduler without aging (Module 03).

LIVELOCK IS THE HARDEST TO DIAGNOSE

A deadlocked system is obviously stuck: CPU use falls to nothing and a debugger shows every thread blocked.

A livelocked system is at full utilisation with every thread actively executing. Monitoring shows a busy, healthy machine doing no work. The classic cause is perfect symmetry: two threads detect a conflict at the same instant, both release their resources, both retry immediately, and they collide again.

The fix is to **break the symmetry** with randomised exponential backoff — each retry waits a random interval, growing with each failure. This is exactly Ethernet's collision resolution, for exactly the same reason, and it works for the same reason: randomness makes repeated collision exponentially unlikely.

5 Rules for real code

- 1 **Define a lock order and document it** in the header where the locks are declared.
- 2 **Hold the fewest locks for the shortest time.** Every additional simultaneously-held lock multiplies the opportunities for a cycle.
- 3 **Never call unknown code while holding a lock.** A callback, a virtual method, or a plugin may acquire any lock at all, including one you hold. This single rule prevents a large fraction of real deadlocks and is the one most often broken.
- 4 **Use trylock with backoff** where a consistent ordering is genuinely unavailable — but prefer fixing the ordering.
- 5 **Enable lockdep or ThreadSanitizer in test builds.** They find ordering violations without requiring the deadlock to occur, which is the only reliable way to catch these before production.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
OSTEP — Chapter 32 (Common Concurrency Problems)	Deadlock and the non-deadlock bugs, with a study of what actually occurs in real code.
Linux kernel documentation — lockdep	How runtime lock-order validation works. Worth reading for the design even if you never use Linux.
Coffman, Elphick & Shoshani — System Deadlocks (1971)	The paper that identified the four conditions.
xv6 book — Chapter 6 (Locking), deadlock discussion	xv6's lock ordering and why it is documented where it is.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Write a program with two threads and two locks that deadlocks reproducibly. Attach a debugger and inspect both threads' stacks.
- 2 Fix it with address-based lock ordering. Verify by running it a million times.
- 3 Implement a bank transfer function for N accounts with M threads doing random transfers. Demonstrate the deadlock without ordering and its absence with ordering.
- 4 Implement a wait-for graph and a cycle detector. Run it periodically against a deadlocking program and report the cycle it finds.
- 5 Implement a lock-order validator: record the highest-level lock each thread holds and assert on violations. Test it against deliberately wrong code.
- 6 Construct a livelock: two threads that each acquire one lock, detect the other is held, release, and retry immediately. Observe 100% CPU with no progress, then fix it with randomised backoff.
- 7 Construct starvation: a reader-writer lock under continuous read load where the writer never proceeds. Then implement writer-preference and show it resolves.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 State the four Coffman conditions and give one way to break each.
- 2 Why does global lock ordering make deadlock structurally impossible rather than merely unlikely?
- 3 When would you order locks by address and when by level?
- 4 Why is the Banker's algorithm not used in real operating systems?

- 5 Why can databases use detection and recovery while operating systems largely cannot?
- 6 Distinguish deadlock, livelock, and starvation, and say which is hardest to notice from monitoring.
- 7 Why is 'never call unknown code while holding a lock' such an effective rule?